PII Detection and Pseudonymization Pipeline

BIG DATA SYSTEMS AND RESPONSIBLE AI ENGINEERING

PROJECT REPORT

Team Details

Name	SRN	Section
Nakka Gopya	PES1UG24AM107	B
Devika N	PES1UG24AM080	B

Problem Statement and Objectives

Support logs and similar text data may contain names, email addresses, and phone numbers. Publishing or analysing these logs without protection can expose personal information. The project aims to build a repeatable pipeline that detects common PII categories, substitutes stable pseudonyms, measures detection performance, and examines privacy weaknesses that remain after substitution.

Detect names, email addresses, and phone numbers in text using Presidio and its NLP dependencies.

Process records with Spark and store input/output data in Parquet format.

Use keyed HMAC-based pseudonyms so repeated values can be represented consistently.

Compare baseline and tuned detection metrics and measure Spark throughput.

Study residual risks from missed detections, candidate-based dictionary attacks, and record linkage.

System Design and Implementation

The workflow begins with public or synthetic datasets stored as Parquet. A Spark job applies PII detection to text records, then replaces detected spans with typed pseudonyms such as <EMAIL_ADDRESS_…> and <PERSON_…>. The pseudonymized records are written to a processed Parquet location. Separate evaluation and re-identification modules produce metrics, tables, and visualizations.

Component	**Role**
Apache Spark / PySpark	Distributed data processing and throughput experiments
Microsoft Presidio + spaCy	Recognition of PII entities in text
HMAC-SHA256	Keyed deterministic pseudonym generation
Parquet / pandas	Storage and metric handling
scikit-learn / Matplotlib	Evaluation support and result visualizations
pytest	Automated regression checks

The secret key is supplied through the PSEUDO_SECRET environment variable. The pipeline was updated to fail if the variable is absent rather than silently using a predictable default. Docker Compose was also updated to

require the variable. A real deployment should use a secret manager or protected environment configuration, not a secret committed to source control.

Methodology

The project was carried out in several stages, starting with data preparation and PII detection, followed by pseudonymization, performance evaluation, and privacy-risk analysis.

1. Data Collection and Preparation

Publicly available datasets, including ai4privacy and Bitext, were used for the experiments. The data was stored in Parquet format for efficient processing. The detection evaluation used 1,000 English documents containing 654 annotated PII entities.

2. PII Detection

Microsoft Presidio was used to detect person names, phone numbers, and email addresses. Presidio uses named-entity recognition, pattern matching, and other recognizers to identify sensitive information in text. The detected entities were categorized by type so that their performance could be evaluated separately.

We compared baseline and tuned configurations using precision, recall, and F1-score to measure detection performance and identify areas for improvement.

3. Pseudonymization

After detecting PII, the pipeline replaced the identified values with pseudonyms generated using HMAC-SHA256 and a secret key. The same input value produces the same pseudonym when the key and entity type remain unchanged. This makes it possible to maintain consistency across multiple records without displaying the original identifier. The secret key is supplied through an environment variable rather than being hard-coded into the source code.

4. Distributed Processing Using Apache Spark

Apache Spark was used to process records and save the pseudonymized output in Parquet format. The pipeline reads the input data, applies PII detection and replacement, and saves the processed records in Parquet format.

Throughput was measured on 10,000 records using one-core and two-core configurations, with three runs for each configuration.

5. Performance and Privacy Evaluation

The baseline and tuned configurations were compared using detection metrics and the number of missed PII entities. Three privacy experiments were also conducted: a dictionary attack, missed-PII analysis, and record-linkage analysis.

- **Dictionary attack:** We tested whether candidate values could be matched to generated pseudonyms when the correct secret key or an incorrect key was used.
- **Missed-PII analysis:** We examined the entities that the detector failed to identify. These missed entities represent a potential source of information leakage because they may remain unchanged in the output.
- **Record-linkage analysis:** We examined whether records could be matched using additional information, even after direct identifiers had been replaced.

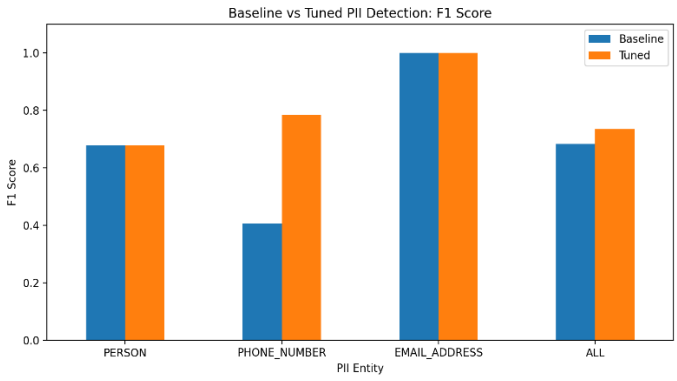
These examined whether information could still be exposed or linked after pseudonymization.

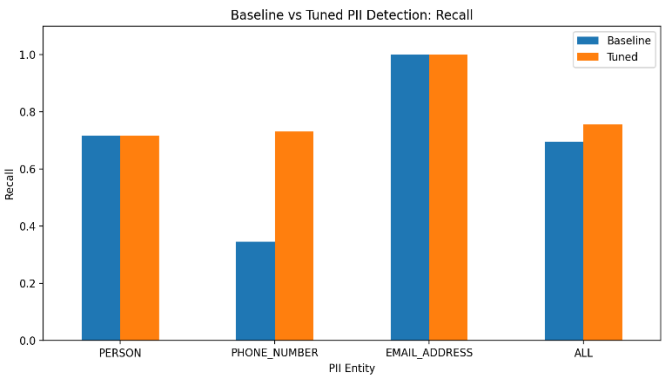
6. Testing and Validation

The automated test suite completed with 15 passing tests. An end-to-end test using five synthetic records verified that email addresses were replaced consistently. The results were saved as CSV files and graphs for analysis and reporting.

Results: PII Detection

Metric	Baseline	Tuned
Overall F1-score	0.6837	0.7344
Email F1-score	1.0000	1.0000
Person F1-score	0.6776	0.6776
Phone-number F1-score	0.4068	0.7835
Missed annotated entities	199 / 654	159 / 654
Miss rate	30.4%	24.3%





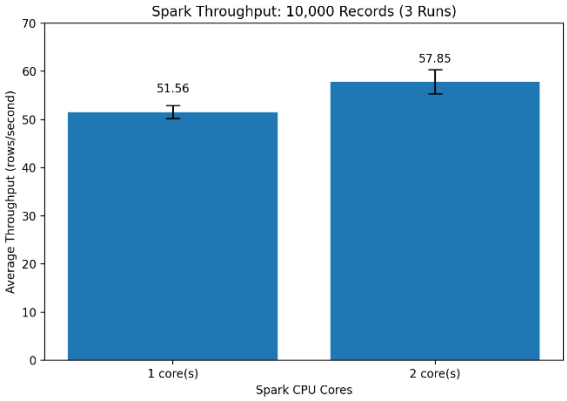
Baseline vs. Tuned PII Detection

The baseline and tuned PII detection configurations were compared using F1-score and recall across three categories: person names, phone numbers, and email addresses. The results show that tuning improved overall detection performance, with the most substantial improvement observed in phone-number detection. The phone-number F1-score increased from approximately 0.41 to 0.78, while recall increased from approximately 0.34 to 0.73. This indicates that the tuned configuration identified a greater proportion of actual phone numbers and achieved a better balance between precision and recall.

Person-name detection showed little change between the two configurations, suggesting that further improvements may require additional rules, better training data, or more effective entity-recognition techniques. Both configurations achieved an F1-score and recall of 1.00 for email addresses on the evaluated dataset. Overall, the tuned configuration demonstrated better PII detection performance, particularly for phone numbers. Improved recall is especially important for privacy protection because undetected personal information may remain in the processed text.

Results: Spark Throughput

Configuration	Average throughput
1 Spark core	51.56 rows/second
2 Spark cores	57.85 rows/second



The two-core configuration improved average throughput by approximately 12.2% (about 1.12×). The gain is positive but not linear with the number of cores. Runtime overhead, task scheduling, NLP processing, data volume, and the local execution environment can all affect scaling. These measurements should therefore be reported with their hardware and configuration and should not be generalized to a cluster without further tests.

Results: Privacy-Risk Experiments

Dictionary attack

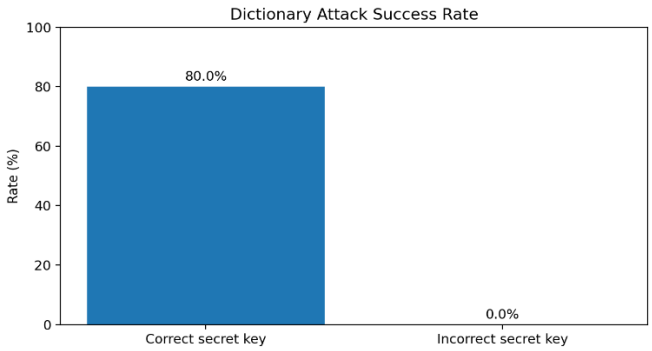
In the small candidate-based experiment, 4 of 5 values (80%) were recovered when the correct secret was available, while 0 of 5 (0%) were recovered with an incorrect secret. This illustrates that an attacker who obtains the key and has candidate inputs may recompute matching pseudonyms. It is not evidence that HMAC was cryptographically cracked.

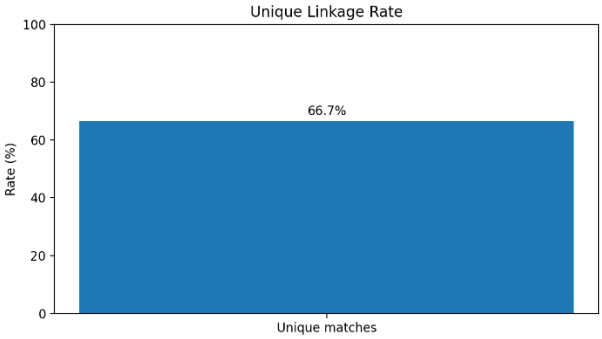
Missed-PII analysis

The tuned detector missed 159 of 654 annotated entities (24.31%), compared with 199 of 654 (30.43%) for the baseline. Any missed identifier can remain in the text and may be exposed downstream. The miss rate is a useful residual-risk indicator, but it is not itself a measured probability of a real-world privacy breach.

Record linkage

In a constructed three-record example, two records produced unique matches and one remained ambiguous, giving a unique-match rate of 66.67% (2/3). This demonstrates the concept that auxiliary attributes can make records linkable even after direct identifiers are replaced. The sample is too small to estimate real-world re-identification risk.





Validation and Testing

The automated test suite completed with 15 passing tests in 1.98 seconds after the secret-key requirement was introduced. An end-to-end Spark check processed five synthetic records and verified that all expected email addresses were replaced, repeated instances of the same email received the same token, and different test email addresses received distinct tokens. These checks provide evidence of functional correctness for the tested cases, but do not prove complete PII coverage or collision-free behavior for all possible inputs.

Limitations

- PII detection is imperfect; names, phone numbers, unusual formats, and contextual identifiers may be missed.
- Deterministic pseudonyms preserve link ability for repeated values and are not equivalent to anonymization.
- Exposure of the HMAC key can enable candidate matching; key rotation and access control are important.
- Truncated HMAC tokens have a theoretical collision risk.

Conclusion

In this project, we developed a pipeline that detects personal information in text and replaces it with pseudonyms to help protect sensitive data. We used Microsoft Presidio for PII detection, Apache Spark for data processing, and HMAC-SHA256 to generate consistent pseudonyms.

The results showed that tuning improved the overall F1-score from 0.6837 to 0.7344 and reduced the number of missed PII entities from 199 to 159. Phone-number detection showed the biggest improvement, while person-name detection still needs further work. We also found that processing with two Spark cores was faster than using one core, increasing throughput from 51.56 to 57.85 rows per second.

Our privacy experiments showed that replacing personal information does not completely eliminate privacy risks. Some PII may go undetected, and additional information or access to the secret key could make it possible to identify records. Overall, this project helped us understand how to combine data processing and privacy protection while measuring the system's performance and limitations. In the future, we can improve detection accuracy, test the pipeline on larger datasets, and strengthen its security to make it more reliable for real-world use.